ASSIGNMENT 02

Object Oriented Programming with C++ (DSA0104)

Part 1: Questions Q1 to Q4

Name	Sangeet TR
Reg No	1921260245
Course Name	Object Oriented Programming with C++
Course Code	DSA0104
GitHub Repository	https://github.com/Sangeetthangaraj/DSA0104---object-oriented-programming-with-c-

Contents and marks

Q	Topic	Marks	Source folder
1	Call by value, by reference and by pointer	6	Q1/
2	Return by reference and its dangers	4	Q2/
3	Default arguments and function overloading	6	Q3/
4	Specifying a class with an array inside it	6	Q4/

How the evidence was produced

- Every program was compiled with `g++ 13.3.0 -std=c++17 -Wall -Wextra` on Ubuntu 24.04 and executed. Compiler messages and program output in this report are copied verbatim from the files in `outputs/` of the GitHub repository (written by `run_all.sh`).
- Predictions were written from reasoning about the code before the program was run; each is compared with the observed result next to it.
- Timings come from a single-core virtual machine, so absolute times will differ on another machine. Only ratios between variants should be compared.
- This is Part 1 of 2. Part 2 covers Q4 to Q7 (Q4 is repeated there because Q7 builds on it). The source code of all programs is in the GitHub repository.

Q1 - Call by Value, by Reference and by Pointer (6 marks)

1.1 The problem and the three implementations

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to a student's fee. The original function printed the right fee **inside** the function but the stored fee never changed. That symptom is exactly what call by value produces: the function reduced its own private copy.

`q1_three_ways.cpp` implements the concession three ways. Each function also prints whether the object it is working on is the caller's own variable (by comparing addresses).

q1_three_ways.cpp

```
// Q1 - Hostel Fee Concession Updater: by value, by reference, by pointer,
// plus the "return the new fee" design and a const-reference example.
#include <iostream>
#include <string>

const int MERIT = 10, SPORTS = 15, SINGLE_PARENT = 25;
const void* callerFeeAddr = nullptr;      // address of the caller's variable

// 1. By value: 'fee' is a new local copy on the function's stack frame.
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "  [value]   inside function fee = " << fee
              << ", same object as caller? "
              << (&fee == callerFeeAddr ? "yes" : "no") << '\n';
}
// 2. By reference: 'fee' is another name for the caller's variable.
void applyByReference(long long& fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "  [ref]     inside function fee = " << fee
              << ", same object as caller? "
              << (&fee == callerFeeAddr ? "yes" : "no") << '\n';
}
// 3. By pointer: 'p' is a local copy of an ADDRESS; *p is the caller's fee.
void applyByPointer(long long* p, int percent)
{
    if (!p) return;                       // a pointer can be null - check it
    *p -= *p * percent / 100;
    std::cout << "  [pointer] inside function *p  = " << *p
              << ", p points at caller's object? "
              << (p == callerFeeAddr ? "yes" : "no")
              << ", &p (the pointer itself) is the caller's? "
              << ((const void*)&p == callerFeeAddr ? "yes" : "no") << '\n';
}
// 4. Return the new fee (pure function): caller decides what to do with it.
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}
// 5. const reference: read-only access without copying a large object.
struct Student {
    std::string name; long long fee; char history[1024];   // "large" object
};
long long feeAfter(const Student& s, int percent)
{
    // s.fee -= 1;   // would not compile: s is const
    return s.fee - s.fee * percent / 100;
}

int main()
{
    std::cout << "sizeof(long long) = " << sizeof(long long)
              << ", sizeof(Student) = " << sizeof(Student) << '\n';

    long long fee = 45000;
    callerFeeAddr = &fee;
    std::cout << "Prediction: value->45000, reference->40500, pointer->36450\n";
    applyByValue(fee, MERIT);
    std::cout << "after by value   : " << fee << '\n';
    applyByReference(fee, MERIT);
    std::cout << "after by ref     : " << fee << '\n';
    applyByPointer(&fee, MERIT);
    std::cout << "after by pointer : " << fee << '\n';
```

```
    std::cout << "\nChained concessions (merit, sports, single parent), "
                 "return-value design:\n";
    long long f = 45000;
    f = applyConcession(f, MERIT);          // 40500
    f = applyConcession(f, SPORTS);         // 34425
    f = applyConcession(f, SINGLE_PARENT);  // 25819
    std::cout << "  45000 -> " << f << '\n';

    std::cout << "\nSame chain by reference (in place):\n";
    long long g = 45000;
    callerFeeAddr = &g;
    applyByReference(g, MERIT);
    applyByReference(g, SPORTS);
    applyByReference(g, SINGLE_PARENT);
    std::cout << "  45000 -> " << g << '\n';

    Student s{"Asha", 45000, {0}};
    std::cout << "\nconst-reference read-only preview: " << feeAfter(s, MERIT)
              << " (stored fee still " << s.fee << ")\n";
}
```

1.2 Predictions before running, and observed results

Starting fee 45000 and a 10% concession, applied in this order: value, then reference, then pointer.

Call	Predicted stored fee	Observed stored fee	Reason
by value	45000	45000	function reduced its copy (40500 inside), copy destroyed on return
by reference	40500	40500	reference is another name for the caller's variable: 45000 - 4500
by pointer	36450	36450	acts on the caller's variable, now 40500: 40500 - 4050

Captured output of q1_three_ways

```
sizeof(long long) = 8, sizeof(Student) = 1064
Prediction: value->45000, reference->40500, pointer->36450
  [value]   inside function fee = 40500, same object as caller? no
after by value   : 45000
  [ref]     inside function fee = 40500, same object as caller? yes
after by ref     : 40500
  [pointer] inside function *p  = 36450, p points at caller's object? yes, &p (the pointer
itself) is the caller's? no
after by pointer : 36450

Chained concessions (merit, sports, single parent), return-value design:
  45000 -> 25819

Same chain by reference (in place):
  [ref]     inside function fee = 40500, same object as caller? yes
  [ref]     inside function fee = 34425, same object as caller? yes
  [ref]     inside function fee = 25819, same object as caller? yes
  45000 -> 25819

const-reference read-only preview: 40500 (stored fee still 45000)
exit status: 0
```

There is no difference between prediction and observation. The "same object as caller?" lines confirm the mechanism: **no** for the value copy, **yes** for the reference, and for the pointer the pointer variable `p` itself is a different object from the caller's fee, but the address stored inside it is the caller's fee.

1.3 Inquiry

Where each version stores its parameter

Version	What exists inside the function	Size (64-bit)
by value	A new `long long` object in the function's stack frame, initialised with a copy of the argument.	8 bytes (sizeof(long long) = 8)
by reference	No new fee object. The name `fee` is an alias of the caller's object. The compiler usually passes the address internally (or removes it completely when the call is inlined).	0 new objects (an address at most)
by pointer	A new local **pointer** variable `p` in the stack frame holding the address of the caller's fee. `*p` is the caller's object.	8 bytes for the pointer

Why only two of them change the caller's fee

Assignment changes the object that the name refers to. In the value version that object is the callee's own copy, which ceases to exist at the closing brace. In the reference and pointer versions the name (or the address) leads to the caller's object, so the same `fee -= ...` modifies the original. Passing by value is therefore not "wrong"; it is the right choice when the function must **not** change the caller's data.

When a const reference parameter should be used

Use `const T&` when the function only **reads** the argument and the type is expensive to copy (a `Student` with strings or arrays, a `std::string`, a `std::vector`). It avoids the copy, the compiler rejects any write through it, and it can bind to temporaries. For small built-in types (`int`, `long long`, `double`) pass by value instead: copying 8 bytes is as cheap as passing an address, and there is no indirection. In the program, `feeAfter(const Student&, int)` previews a fee without copying the 1064-byte `Student` and leaves the stored fee at 45000 (last line of the output above).

1.4 Predict the Output

q1_predict.cpp

```
// Q1 - Predict the Output (program given in the question, unchanged)
#include <iostream>
void change(int a, int& b, int* c)
{
    a += 10;
    b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

My predicted output (written before running)

```
1 12 13
1 100
16 13 6
```

Reasoning for each line

- **change(a, b, &c):** `a` is passed by value, so `a += 10` changes a copy and `a` stays 1. `b` is a reference, so `b += 10` changes the caller's `b` to 12. `c` is passed as a pointer and `*c += 10` changes the caller's `c` to 13. First line: `1 12 13`.
- **bigger(a, c) = 100:** `bigger` returns an `int&` to the larger of `a` (1) and `c` (13). The condition `p > q` is false, so it returns `q`, which is `c`. The call is an lvalue, so assigning 100 sets `c` to 100. Second line: `1 100`.
- **show(1):** default arguments y = 5 and z = 10 are both used: 1 + 5 + 10 = 16.
- **show(1, 2):** only z = 10 is defaulted: 1 + 2 + 10 = 13.
- **show(1, 2, 3):** no default used: 6.

Observed output

```
1 12 13
1 100
16 13 6
exit status: 0
```

Differences between prediction and actual output: none. One detail worth noting: every call to `show` prints a trailing space, so the third line is `16 13 6 ` followed by the newline; the trailing space is invisible in a terminal. The status line "exit status: 0" is added by `run_all.sh`, not by the program.

1.5 Design Decision: change in place or return the new fee?

Several concessions may apply to one student. **Decision: return the new fee** (`long long applyConcession(long long fee, int percent)`) and let the caller assign it. Defence:

- **Composition is explicit.** `fee = apply(apply(apply(fee, 10), 15), 25)` shows the order of application. Order matters because each step truncates with integer division, and the office must decide whether concessions compound or add. With in-place calls the same information is hidden in a sequence of statements with side effects.
- **No accidental double application.** An in-place function applied twice silently reduces the fee twice. A returning function lets the caller preview a fee (`feeAfter`) and commit it only once, in one visible assignment.
- **Easy to test.** A function with no side effects is tested with plain inputs and outputs.
- **Cost is nothing for a `long long`.** Returning 8 bytes by value is as cheap as passing a reference.

In-place update by reference is still the better design when the concession is a **member function of Student** (`s.applyConcession(10)`), because then the object owns its fee and the update is the whole purpose of the call. Both chains give the same result in this program: 45000 -> 40500 -> 34425 -> 25819 (second section of the captured output above).

1.6 Complete the Code

q1_complete.cpp

```
// Q1 - Complete the Code (TODO 1 completed)
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;   // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
    // TODO 1 (done): reduce the caller's fee by percent, integer arithmetic,
    // call by reference.
    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value    : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Observed output (expected: 45000 after by value, 40500 after by reference)

```
after call by value    : 45000
after call by reference: 40500
exit status: 0
```

Why the call by value leaves the fee at 45000: `applyByValue` receives a copy of `hostelFee`. The statement `fee -= fee * percent / 100` changes only that copy (45000 -> 40500), and the copy is destroyed when the function returns. The caller's `hostelFee` was never touched. The reference version reduces the original once: 45000 - 45000 * 10 / 100 = 40500.

1.7 Alternative concept and complexity analysis

Alternative solution (return the new fee):

```
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}
// caller:  hostelFee = applyConcession(hostelFee, 10);
```

Time is O(1) in every version: one multiplication, one division, one subtraction. The difference is the data copied per call (`sizeof(long long)` = 8 and `sizeof(Student)` = 1064 were printed by the program):

Passing style	long long (8 bytes)	Student (1064 bytes + string heap data)	Can modify caller's object?
by value	8 bytes copied	1064 bytes copied, and the `std::string` member is deep-copied (O(length of name), may allocate)	No
by reference	nothing copied (alias)	nothing copied	Yes
by const reference	nothing copied (not worth it for 8 bytes)	nothing copied	No (compiler enforced)
by pointer	8 bytes (the address)	8 bytes (the address)	Yes (and can be null)
return new fee	8 bytes in, 8 bytes out	only the fee is passed, so pass `s.fee`, not `s`	No (caller assigns)

Space beyond the copies is O(1) in all versions. The cost of by-value copying grows with the size of the object, not with the number of students, so it is O(1) in the complexity sense, but with a large constant for `Student`.

Q2 - Return by Reference and Its Dangers (4 marks)

2.1 Working functions

`q2_return_ref.cpp` keeps the marks in an array. `highest()` returns an `int&` to the highest mark so a moderation of +2 can be applied directly to the element; `highestConst()` returns `const int&`; `highestCopy()` returns a copy. `highestIndex()` and `highestPtr()` are the alternatives of section 2.5.

q2_return_ref.cpp

```
// Q2 - Examination Marks Register: return by reference / const reference / copy
#include <iostream>

// Returns a reference to the highest element (lvalue: can be assigned to).
int& highest(int marks[], int n)
{
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];                    // marks[] outlives the call: safe
}
// Read-only reference: no copy, caller cannot modify the register through it.
const int& highestConst(const int marks[], int n)
{
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
// Copy: caller receives an independent value.
int highestCopy(const int marks[], int n) { return highestConst(marks, n); }

// Alternatives: index and pointer.
int highestIndex(const int marks[], int n)
{
    int best = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[best]) best = i;
    return best;
}
int* highestPtr(int marks[], int n) { return &marks[highestIndex(marks, n)]; }

void print(const int m[], int n)
{
    for (int i = 0; i < n; ++i) std::cout << m[i] << ' ';
    std::cout << '\n';
}

int main()
{
    int marks[6] = {56, 78, 91, 67, 85, 72};
    std::cout << "original            : "; print(marks, 6);

    highest(marks, 6) += 2;                // moderation on the call itself
    std::cout << "after highest()+=2  : "; print(marks, 6);

    highest(marks, 6) = 100;               // call on the left of '='
    std::cout << "after highest()=100 : "; print(marks, 6);

    int copy = highestCopy(marks, 6);      // copy
    copy -= 50;
    const int& cref = highestConst(marks, 6);
    // cref += 2;   // error: assignment of read-only reference
    std::cout << "copy changed to " << copy << ", register max still "
              << cref << '\n';

    marks[0] = 99;                         // a const-reference sees updates
    std::cout << "cref after marks[0]=99, max is now at index "
              << highestIndex(marks, 6) << ", cref still = " << cref << '\n';

    // index and pointer call styles
    marks[highestIndex(marks, 6)] += 2;
    *highestPtr(marks, 6) += 2;
    std::cout << "via index and pointer: "; print(marks, 6);
}
```

Captured output

```
original            : 56 78 91 67 85 72
after highest()+=2  : 56 78 93 67 85 72
after highest()=100 : 56 78 100 67 85 72
copy changed to 50, register max still 100
cref after marks[0]=99, max is now at index 2, cref still = 100
via index and pointer: 99 78 104 67 85 72
exit status: 0
```

What the output shows

- `highest(marks, 6) += 2;` changed the array element 91 to 93. A returned reference is an **alias of the element itself**, so the call can be used as a target of += and =.
- `highest(marks, 6) = 100;` put the call on the left of = and set that element to 100.
- The **copy** (`highestCopy`) was reduced by 50 and the register was unchanged (still 100): a copy is independent.
- The **const reference** `cref` is bound to one specific element (index 2). After `marks[0] = 99` the highest mark is at index 2 and still 100; note that a reference does not "re-search": it stays bound to the element it was bound to. Writing `cref += 2` does not compile.

2.2 Returning a reference to a local variable

q2_dangling.cpp

```
// Q2 - Returning a reference to a LOCAL variable (undefined behaviour).
// Compile with -Wall to see the warning.
#include <iostream>

int& badHighest()
{
    int local = 95;          // lives on this function's stack frame
    return local;            // reference to something that dies at '}'
}
int overwriteStack(int x)    // reuses the dead frame
{
    int pad[16];
    for (int i = 0; i < 16; ++i) pad[i] = x + i;
    return pad[3];
}
int main()
{
    int& r = badHighest();   // r is dangling
    overwriteStack(7);
    std::cout << "value read through dangling reference: " << r << '\n';
}
```

Compiler output, g++ -Wall -Wextra -O0

```
Q2/q2_dangling.cpp: In function 'int& badHighest()':
Q2/q2_dangling.cpp:8:12: warning: reference to local variable 'local' returned [-Wreturn-
local-addr]
    8 |     return local;            // reference to something that dies at '}'
      |            ^~~~~
Q2/q2_dangling.cpp:7:9: note: declared here
    7 |     int local = 95;          // lives on this function's stack frame
      |         ^~~~~
```

Run at -O0

```
Segmentation fault
exit status: 139
```

Run at -O2 (same warning at compile time)

```
Segmentation fault
exit status: 139
```

What the compiler warns about: `reference to local variable 'local' returned [-Wreturn-local-addr]`. **Why using it is undefined behaviour:** `local` has automatic storage duration. Its lifetime ends at the closing brace and its stack space is released for reuse. The returned

reference therefore refers to an object that no longer exists (a dangling reference). Reading it is undefined behaviour: the program may print the old value, print garbage, or crash. On this machine it crashed with a segmentation fault (exit status 139) at both `-O0` and `-O2`. The exact symptom is not guaranteed and may differ on other compilers, which is why the **warning must never be ignored**.

2.3 Return a reference, a const reference or a copy

Return type	What the caller gets	Can modify the register?	Cost	Main risk
`int&`	alias of the stored element; can be used on the left of =	Yes	no copy	dangling if the element is destroyed or moved; unintended modification
`const int&`	read-only alias	No	no copy	dangling; the alias is bound to one element, not to "the highest"
`int` (copy)	independent value	No	copy (cheap for `int`, expensive for large objects)	caller may mistake the copy for the stored mark; changes are lost

Rule of thumb: return by reference only an object that **outlives the call** (an element of caller-owned storage, a member, a static). Never return a reference to a parameter passed by value, a local variable or a temporary.

2.4 Changed condition: the register is a std::vector that may grow

q2_vector.cpp

```
// Q2 - Changed condition: register stored in a growing std::vector.
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> marks = {56, 78, 91};
    marks.reserve(3);                          // capacity exactly 3
    int* oldPtr = &marks[2];                   // same address a reference holds
    std::size_t oldCap = marks.capacity();
    std::cout << "before: size=" << marks.size() << " capacity=" << oldCap
              << "  address of marks[2] saved\n";

    marks.push_back(67);                       // exceeds capacity -> reallocation
    std::cout << "after : size=" << marks.size()
              << " capacity=" << marks.capacity() << '\n';
    std::cout << "storage moved? "
              << (oldPtr != &marks[2] ? "YES - old reference/pointer is dangling"
                                      : "no") << '\n';
    // Using *oldPtr or the old reference now is undefined behaviour.

    // Safe alternative: remember the INDEX and re-resolve it each time.
    std::size_t idx = 2;
    marks.push_back(70); marks.push_back(80);
    marks[idx] += 2;
    std::cout << "marks[" << idx << "] via index after growth = " << marks[idx] << '\n';
}
```

Captured output

```
before: size=3 capacity=3  address of marks[2] saved
after : size=4 capacity=6
storage moved? YES - old reference/pointer is dangling
marks[2] via index after growth = 93
exit status: 0
```

A `std::vector` keeps its elements in one contiguous block. When `push_back` exceeds the **capacity**, the vector allocates a larger block, moves the elements into it and frees the old block. Here the capacity went from 3 to 6 and the address of `marks[2]` changed. Any reference, pointer or iterator obtained

earlier still points into the **freed** block, so using it is undefined behaviour, the same situation as a dangling reference to a local variable. (An insertion or erase in the middle also invalidates references to the elements after that position, even without reallocation.)

Safe responses: remember the **index** and re-resolve `marks[idx]` each time (shown in the program: 91 + 2 = 93 after growth); call `reserve()` once if the maximum size is known; or re-fetch the reference after every operation that can grow the vector. Do not store references into a growing vector across such operations.

2.5 Alternative concept and complexity analysis: index or pointer

Return type	Calling code (moderation of +2)	Time	Space	Safety after the array becomes a growing vector
`int&`	`highest(marks, n) += 2;`	O(n), one pass	O(1)	Unsafe after reallocation; no way to express "no element" (empty register is undefined behaviour)
`int` index	`marks[highestIndex(marks, n)] += 2;`	O(n), one pass	O(1)	Safe across reallocation (re-resolved on every use); must be bounds-checked; a return of -1 can mean "empty"
`int*`	`*highestPtr(marks, n) += 2;`	O(n), one pass	O(1)	Unsafe after reallocation (stale address); can return `nullptr` for empty, which the caller must check before dereferencing

All three scan the array once, so time is O(n) and extra space is O(1) (one index variable). The only difference is **safety and convenience**: the reference gives the cleanest call but is the most fragile; the **index is the safest design** for a register that will become a growing `std::vector`; the pointer sits between them and is the only one that makes "not found" explicit without an exception. All three functions assume the array is not empty: with `n = 0` the reference version would be undefined behaviour, so a real version needs a precondition or a check.

Q3 - Default Arguments and Function Overloading (6 marks)

3.1 The ambiguity in the original program

q3_ambiguous.cpp

```
// Q3 - the developer's original three functions. DOES NOT COMPILE.
#include <iostream>
double charge(double weight)                       { return weight * 50; }
double charge(double weight, bool express)         { return weight * 50 + (express ? 40 : 0);
}
double charge(double weight, bool express = false, bool insured = false)
{ return weight * 50 + (express ? 40 : 0) + (insured ? 20 : 0); }

int main()
{
    std::cout << charge(2.0) << '\n';          // 1-arg or 3rd with defaults?
    std::cout << charge(2.0, true) << '\n';    // 2-arg or 3rd with default?
}
```

Compiler output (errors)

```
Q3/q3_ambiguous.cpp: In function 'int main()':
Q3/q3_ambiguous.cpp:10:24: error: call of overloaded 'charge(double)' is ambiguous
   10 |     std::cout << charge(2.0) << '\n';          // 1-arg or 3rd with defaults?
      |                  ~~~~~~^~~~~
Q3/q3_ambiguous.cpp:3:8: note: candidate: 'double charge(double)'
    3 | double charge(double weight)                       { return weight * 50; }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
Q3/q3_ambiguous.cpp:11:24: error: call of overloaded 'charge(double, bool)' is ambiguous
   11 |     std::cout << charge(2.0, true) << '\n';    // 2-arg or 3rd with default?
      |                  ~~~~~~^~~~~~~~~~~
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool)'
    4 | double charge(double weight, bool express)         { return weight * 50 + (express ?
40 : 0); }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
compile exit status: 1
```

Explanation. For `charge(2.0)` two functions are viable: `charge(double)` and `charge(double, bool = false, bool = false)`. The argument 2.0 is an exact match for the `double` parameter in both. **Default arguments do not take part in ranking**, so neither function is better and the call is ambiguous. In the same way `charge(2.0, true)` matches `charge(double, bool)` and the three-parameter version equally well. The first two functions are therefore redundant: the third already covers one, two and three arguments.

3.2 Redesign: short calls, three kinds of input, no ambiguity

Design rules used: (1) **one** function per kind of weight, with the defaults placed on that function only, so no overload is a prefix of another; (2) overloads differ in the **type of the first argument**: `double` (kg), `int` (grams) and `const Parcel&`; (3) the optional flags are defaulted from the right.

q3_charge.cpp

```
// Q3 - Courier Charge Calculator: defaults + overloading, no ambiguity.
#include <iostream>
#include <iomanip>

struct Parcel { double kg; bool express; bool insured; };

const double RATE_PER_KG = 50.0, EXPRESS_FEE = 40.0, INSURANCE_RATE = 0.02;

// (1) weight in kilograms (double) - the one real implementation.
double charge(double kg, bool express = false, bool insured = false)
{
    std::cout << "  [kg version]     ";
    double base = kg * RATE_PER_KG;
    return base + (express ? EXPRESS_FEE : 0.0) + (insured ? base * INSURANCE_RATE : 0.0);
```

```
}
// (2) weight in grams (int).
double charge(int grams, bool express = false, bool insured = false)
{
    std::cout << "  [grams version]  ";
    double base = (grams / 1000.0) * RATE_PER_KG;
    return base + (express ? EXPRESS_FEE : 0.0) + (insured ? base * INSURANCE_RATE : 0.0);
}
// (3) a Parcel object.
double charge(const Parcel& p)
{
    std::cout << "  [Parcel version] ";
    double base = p.kg * RATE_PER_KG;
    return base + (p.express ? EXPRESS_FEE : 0.0) + (p.insured ? base * INSURANCE_RATE :
0.0);
}

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Predictions: 2.5 ->kg | (2.5,true) ->kg | 1500 ->grams | 2 ->grams |"
              " 2.0f ->kg | Parcel ->Parcel\n";
    Parcel p{3.0, true, true};
    std::cout << "charge(2.5)          = " << charge(2.5)          << '\n';
    std::cout << "charge(2.5, true)    = " << charge(2.5, true)    << '\n';
    std::cout << "charge(1500)         = " << charge(1500)         << '\n';
    std::cout << "charge(2)            = " << charge(2)            << '\n';
    std::cout << "charge(2.0f)         = " << charge(2.0f)         << '\n';
    std::cout << "charge(p)            = " << charge(p)            << '\n';
    std::cout << "charge(500,true,true)= " << charge(500, true, true) << '\n';
}
```

Predictions made before running (six calls plus one extra)

Call	**Predicted function**	**Reason (rank of conversion)**	**Predicted charge**
`charge(2.5)`	kg (double)	exact match for `double`; `int` would need a narrowing conversion	125.00
`charge(2.5, true)`	kg (double)	exact match, express flag given	165.00
`charge(1500)`	grams (int)	exact match for `int`	75.00
`charge(2)`	grams (int)	exact match for `int` beats int->double conversion; this means **2 grams**	0.10
`charge(2.0f)`	kg (double)	`float->double` is a **promotion**, which ranks above the `float->int` conversion	100.00
`charge(p)` (Parcel 3 kg, express, insured)	Parcel	only viable candidate	150 + 40 + 3 = 193.00
`charge(500, true, true)`	grams (int)	exact match for `int`, two defaults overridden	25 + 40 + 0.50 = 65.50

Observed output

```
Predictions: 2.5 ->kg | (2.5,true) ->kg | 1500 ->grams | 2 ->grams | 2.0f ->kg | Parcel -
>Parcel
charge(2.5)          =   [kg version]     125.00
charge(2.5, true)    =   [kg version]     165.00
charge(1500)         =   [grams version]  75.00
charge(2)            =   [grams version]  0.10
charge(2.0f)         =   [kg version]     100.00
charge(p)            =   [Parcel version] 193.00
charge(500,true,true)=   [grams version]  65.50
exit status: 0
```

Observed selections match all predictions. The text `[kg version]` appears after = because C++17 evaluates the left operand of << before the right operand: the label is printed first, then `charge(...)` runs (printing its trace) and then its result is printed.

A design weakness the experiment exposed: `charge(2)` selects the **grams** function, so a user who meant 2 kg is charged 0.10. Exact-match rules are working correctly; the interface is the problem. Fixes: give the functions different names (`chargeKg`, `chargeGrams`), or wrap the weights in distinct types (`Kg`, `Grams`). A second consequence is shown below.

q3_long_ambiguous.cpp

```
// Q3 - even the redesigned set is ambiguous for a 'long' argument. DOES NOT COMPILE.
#include <iostream>
double charge(double kg, bool express = false, bool insured = false) { return kg; }
double charge(int grams, bool express = false, bool insured = false) { return grams; }
int main() { std::cout << charge(2L) << '\n'; }   // long->int and long->double are both
conversions
```

Compiler output for charge(2L)

```
Q3/q3_long_ambiguous.cpp: In function 'double charge(double, bool, bool)':
Q3/q3_long_ambiguous.cpp:3:31: warning: unused parameter 'express' [-Wunused-parameter]
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg; }
      |                          ~~~~~^~~~~~~~~~~~~~~
Q3/q3_long_ambiguous.cpp:3:53: warning: unused parameter 'insured' [-Wunused-parameter]
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg; }
      |                                                ~~~~~^~~~~~~~~~~~~~~
Q3/q3_long_ambiguous.cpp: In function 'double charge(int, bool, bool)':
Q3/q3_long_ambiguous.cpp:4:31: warning: unused parameter 'express' [-Wunused-parameter]
    4 | double charge(int grams, bool express = false, bool insured = false) { return grams;
}
      |                          ~~~~~^~~~~~~~~~~~~~~
Q3/q3_long_ambiguous.cpp:4:53: warning: unused parameter 'insured' [-Wunused-parameter]
    4 | double charge(int grams, bool express = false, bool insured = false) { return grams;
}
      |                                                ~~~~~^~~~~~~~~~~~~~~
Q3/q3_long_ambiguous.cpp: In function 'int main()':
Q3/q3_long_ambiguous.cpp:5:33: error: call of overloaded 'charge(long int)' is ambiguous
    5 | int main() { std::cout << charge(2L) << '\n'; }   // long->int and long->double are
both conversions
      |                           ~~~~~~^~~~
Q3/q3_long_ambiguous.cpp:3:8: note: candidate: 'double charge(double, bool, bool)'
    3 | double charge(double kg, bool express = false, bool insured = false) { return kg; }
      |        ^~~~~~
Q3/q3_long_ambiguous.cpp:4:8: note: candidate: 'double charge(int, bool, bool)'
    4 | double charge(int grams, bool express = false, bool insured = false) { return grams;
}
      |        ^~~~~~
compile exit status: 1
```

A `long` argument converts to both `int` and `double` with the same rank (conversion), so the call is ambiguous even in the redesigned set. The caller must cast: `charge(2L)` fails, `charge(static_cast<double>(2L))` works.

3.3 Inquiry

How the compiler chooses among overloaded functions

1. **Candidate functions:** all functions with that name visible at the call.
2. **Viable functions:** those whose parameters can accept the arguments (correct count, taking default arguments into account, and an implicit conversion exists for each argument).
3. **Best viable function:** for each argument the conversion is ranked: **exact match** (identity, lvalue-to-rvalue, adding const) is better than **promotion** (`float->double`, `char/short/bool->int`), which is better than **standard conversion** (`int->double`, `double->int`, `long->int`, `int->bool`), which is better than a user-defined conversion. The chosen function must be at least as good as every other for all arguments and strictly better for at least one. If no function wins, the call is **ambiguous** and does not compile. The return type is **not** considered.

This is done entirely at compile time: no run-time cost.

Why default arguments must be given from the right

Arguments are matched to parameters by position, left to right. A caller can omit only a **trailing** group of arguments, so only trailing parameters can have defaults. If `f(int a = 1, int b)` were allowed,

`f(5)` could not tell whether 5 is `a` or `b`, and there is no syntax for leaving a gap such as `f(, 7)`. Therefore once a parameter has a default, every parameter after it must have one too. The default is written once (in the declaration, not repeated in the definition) and is substituted at the call site.

3.4 Alternative concept and complexity analysis

Alternative 1: a single `Parcel` structure with optional fields (`q3_alt_parcel.cpp`):

q3_alt_parcel.cpp

```
// Q3 - Alternative: one Parcel structure with optional fields (no overloads),
// and a function template version.
#include <iostream>
#include <iomanip>

struct Parcel {
    double kg = 0;
    bool express = false;      // optional, default off
    bool insured = false;      // optional, default off
    double declaredValue = 0;  // NEW option: added without touching any signature
};

double charge(const Parcel& p)
{
    double base = p.kg * 50.0;
    return base + (p.express ? 40.0 : 0.0) + (p.insured ? p.declaredValue * 0.01 : 0.0);
}

template <class W>                    // weight in any arithmetic unit, in kg
double chargeT(W weightKg, bool express = false)
{
    return static_cast<double>(weightKg) * 50.0 + (express ? 40.0 : 0.0);
}

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    Parcel a; a.kg = 2.5;
    Parcel b; b.kg = 2.5; b.express = true;
    Parcel c; c.kg = 3; c.insured = true; c.declaredValue = 10000;
    std::cout << charge(a) << ' ' << charge(b) << ' ' << charge(c) << '\n';
    std::cout << chargeT(2) << ' ' << chargeT(2.5f, true) << ' ' << chargeT(2L) << '\n';
}
```

Captured output (Parcel version, then the template version)

```
125.00 165.00 250.00
100.00 165.00 100.00
exit status: 0
```

Alternative 2: a function template `template <class W> double chargeT(W weightKg, bool express = false)` accepts `int`, `float`, `long` and `double`, converting to `double` inside. It removes the `charge(2L)` ambiguity (the output shows `chargeT(2L)` = 100.00) but it also accepts types that make no sense, and the unit is always kg.

Design	Overload resolution time	Data copied per call	Adding a new option (e.g. "fragile")
Overloads + default arguments	compile time only, 0 at run time	`double`/`int` 8 or 4 bytes + 2 bool; Parcel by const reference = 8 bytes (address)	new parameter on every overload; positional calls such as `charge(2.5, false, false, true)` become unreadable and error-prone; callers recompile
One `Parcel` structure	none (one function)	by value 16 bytes (`kg`, 2 bools, padding) or 24 bytes after adding `declaredValue`; 8 bytes by const reference	add a field with a default value: existing calls unchanged, the new option is named at the call site

Design	Overload resolution time	Data copied per call	Adding a new option (e.g. "fragile")
Function template	compile time (template deduction), 0 at run time	size of the argument type	new parameter again changes the template signature; one instantiation per type means more generated code; worse error messages

Overload resolution costs nothing at run time in any design. The `Parcel` design handles **change** best (fields are named and can be added without touching existing calls) and removes the ambiguity problems, at the cost of slightly longer call sites for the simple case. Overloads with defaults give the shortest common calls (`charge(2.5)`). My recommended final design is: the `Parcel` structure as the single real implementation, with the short `charge(double kg)` and `charge(int grams)` overloads kept only as thin wrappers.

Q4 - Specifying a Class with an Array Inside It (6 marks)

4.1 Class specification

`SavingsAccount` has private data (account number, balance, an array of the last 10 transactions and a count) and public `deposit()`, `withdraw()` and `miniStatement()`. The getters and the constructor are defined **inside** the class; `deposit`, `withdraw`, `miniStatement` and the private helper `record` are defined **outside** with the scope resolution operator `SavingsAccount::`. A withdrawal larger than the balance is rejected and is not recorded.

q4_account.cpp

```
// Q4 - SavingsAccount with a private array of the last MAX_TXN transactions.
// Design 1: shifting array. Some members are defined inside the class,
// some outside with the scope resolution operator.
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };     // 'D' deposit, 'W' withdrawal

class SavingsAccount {
    static const int MAX_TXN = 10;       // change to 50 here and nowhere else
    int         accNo;
    double      balance;
    Transaction txn[MAX_TXN];            // private: no one can corrupt count/order
    int         count;                   // how many valid entries (0..MAX_TXN)

    void record(char type, double amount);         // helper, defined outside
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), count(0) {}

    int    getAccNo()   const { return accNo; }      // defined inside the class
    double getBalance() const { return balance; }

    void deposit(double amount);                     // defined outside
    bool withdraw(double amount);                    // defined outside
    void miniStatement() const;                      // defined outside
};

void SavingsAccount::record(char type, double amount)
{
    if (count == MAX_TXN) {                          // 11th transaction: shift left, O(k)
        for (int i = 1; i < MAX_TXN; ++i) txn[i - 1] = txn[i];
        --count;
    }
    txn[count].type = type;
    txn[count].amount = amount;
    ++count;
}
void SavingsAccount::deposit(double amount)
{
    if (amount <= 0) return;
    balance += amount;
    record('D', amount);
}
bool SavingsAccount::withdraw(double amount)
{
    if (amount <= 0 || amount > balance) return false;   // rejected: not recorded
    balance -= amount;
    record('W', amount);
    return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement for account " << accNo << " (last " << count << ")\n";
    for (int i = 0; i < count; ++i)
        std::cout << "  " << std::setw(2) << i + 1 << ". "
                  << (txn[i].type == 'D' ? "Deposit   " : "Withdrawal")
                  << std::setw(9) << txn[i].amount << '\n';
    std::cout << "  Balance: " << balance << '\n';
}

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; ++i) acc.deposit(100 * i);          // 8 transactions
```

```
    acc.withdraw(250);                                          // 9
    acc.withdraw(1e9);                                          // rejected
    acc.miniStatement();
    std::cout << "--- add 3 more (10th fills the array; 11th and 12th discard the two
oldest) ---\n";
    acc.deposit(1111); acc.withdraw(50); acc.deposit(2222);
    acc.miniStatement();
}
```

Captured output

```
Mini-statement for account 1001 (last 9)
   1. Deposit      100.00
   2. Deposit      200.00
   3. Deposit      300.00
   4. Deposit      400.00
   5. Deposit      500.00
   6. Deposit      600.00
   7. Deposit      700.00
   8. Deposit      800.00
   9. Withdrawal   250.00
  Balance: 8350.00
--- add 3 more (10th fills the array; 11th and 12th discard the two oldest) ---
Mini-statement for account 1001 (last 10)
   1. Deposit      300.00
   2. Deposit      400.00
   3. Deposit      500.00
   4. Deposit      600.00
   5. Deposit      700.00
   6. Deposit      800.00
   7. Withdrawal   250.00
   8. Deposit     1111.00
   9. Withdrawal    50.00
  10. Deposit     2222.00
  Balance: 11633.00
exit status: 0
```

4.2 Inquiry

Why the array must be private

The class has an **invariant**: `0 <= count <= MAX_TXN`, entries are in time order, and the balance equals the opening balance plus the recorded history. If `txn` and `count` were public, any code could overwrite a transaction, change `count` so that `miniStatement()` reads beyond the array, or change the balance without a record, bypassing the check in `withdraw`. Keeping the data private means only the three public functions can change it, and the **representation can be replaced** without touching any caller. Q7 does exactly that (shift array -> circular index) and the calling code in `main` does not change.

What happens when the 11th transaction arrives

	Shift the array	**Circular index**
Action	copy `txn[1..9]` to `txn[0..8]`, then write the new transaction at `txn[9]`	write at `(start + count) % 10`, then advance `start` by one (the oldest slot is overwritten)
Elements moved	9 (K - 1) on every add after the array is full	0
Cost per add	O(K)	O(1)
Statement order	physical order = time order, simple loop	physical order is rotated; read through `(start + i) % K`

Observed: after 12 transactions (the 9 shown first plus 3 more) the two oldest (deposits 100 and 200) were discarded and the list starts at 300. The shifting version, the circular version and the `std::deque` version printed **identical** statements (checked with `diff`; see 4.4). The measured cost of the two methods is in Q7.

4.3 Design Decision: the last 50 transactions

Decision: use the circular index. In both designs the capacity is the single constant `MAX_TXN`, so changing 10 to 50 is a one-line edit and no other code changes (loop bounds and the modulus are expressed with it). The difference is in what the change does to the running cost: with shifting, each add after the array is full now moves 49 elements instead of 9 (5.4 times more work); with the circular index the cost of an add stays one write. Memory grows in both from 160 bytes to 800 bytes per account (16 bytes per `Transaction` x 50), because an array always reserves its full size. The public interface, the callers and the tests are unchanged either way. If the ATM should print only part of the history, `miniStatement` can take a `lastN` parameter (done in `q7_redesign.cpp`).

4.4 Alternative concept and complexity analysis

Both alternatives keep the same public interface. The circular buffer:

q4_circular.cpp

```
// Q4 - Design 2: circular index (start + count). Same public interface.
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };

class SavingsAccount {
    static const int MAX_TXN = 10;
    int accNo; double balance;
    Transaction txn[MAX_TXN];
    int start;        // index of the OLDEST transaction
    int count;
    void record(char type, double amount)
    {
        int pos = (start + count) % MAX_TXN;           // where the newest goes
        txn[pos] = {type, amount};
        if (count < MAX_TXN) ++count;
        else start = (start + 1) % MAX_TXN;            // overwrite oldest: O(1)
    }
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), start(0), count(0)
{}
    void deposit(double a) { if (a <= 0) return; balance += a; record('D', a); }
    bool withdraw(double a) { if (a <= 0 || a > balance) return false; balance -= a;
record('W', a); return true; }
    void miniStatement() const
    {
        std::cout << "Mini-statement for account " << accNo << " (last " << count << ")\n";
        for (int i = 0; i < count; ++i) {
            const Transaction& t = txn[(start + i) % MAX_TXN];   // oldest -> newest
            std::cout << "  " << std::setw(2) << i + 1 << ". "
                      << (t.type == 'D' ? "Deposit   " : "Withdrawal")
                      << std::setw(9) << t.amount << '\n';
        }
        std::cout << "  Balance: " << balance << '\n';
    }
};

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; ++i) acc.deposit(100 * i);
    acc.withdraw(250);
    acc.withdraw(1e9);
    acc.deposit(1111); acc.withdraw(50); acc.deposit(2222);
    acc.miniStatement();
}
```

The deque version replaces the array by `std::deque<Transaction>`: `record` becomes `if (txn.size() == MAX_TXN) txn.pop_front(); txn.push_back({t, a});` (full listing in `Q4/q4_deque.cpp`).

Captured output of q4_circular (identical to q4_deque; both equal the second statement of q4_account)

```
Mini-statement for account 1001 (last 10)
   1. Deposit      300.00
   2. Deposit      400.00
   3. Deposit      500.00
   4. Deposit      600.00
   5. Deposit      700.00
   6. Deposit      800.00
   7. Withdrawal   250.00
   8. Deposit     1111.00
   9. Withdrawal    50.00
  10. Deposit     2222.00
  Balance: 11633.00
exit status: 0
```

Design	**Time to add**	**Space**	**Remarks**
Shifting array	O(k): k = number of stored entries moved (K - 1 once full)	K x 16 bytes, plus `count`	simplest code; in-order storage
Circular buffer (start index)	O(1)	K x 16 bytes, plus `start` and `count`	fixed memory, no heap; index arithmetic
`std::deque` (push_back, pop_front)	O(1) amortised	up to K entries in heap blocks, plus bookkeeping (the deque object alone is 80 bytes on this compiler, before any element is stored)	not contiguous; heap allocation; capacity can grow without changing the class

The circular buffer has the best time and the fixed, predictable memory of an array. The deque is the most flexible but costs more per account in memory and allocations.